COMMON COURSE ASSIGNMENT FORMAT

HYDERABAD MOHAMMED NAWAZ-(192325060)

Assignment Information

Particular	Details
Department:	Department of Artificial Intelligence and Data Science
Programme:	B.Tech. — Bachelor of Technology (Artificial Intelligence and Data Science)
Course Code & Course Name	CSA1521 — Cloud Computing and Big Data Analytics
Academic Year / Batch	2025–2026, Batch 192xxxxxx
Faculty Name	Dr. K. Santhiya Lakshmi
Assignment Title	"Smart Network Security Operations Centre Using Cloud-Based Packet Analysis and ML Threat Detection"
Date of Issue	01 September 2026
Date of Submission	01 September 2026
Maximum Marks	100
Course Outcome(s) – CO	Design and implement a cloud-based, intelligent security monitoring solution applying big-data analytics and machine-learning techniques to real-time threat detection
Bloom's Taxonomy Level	L3 – L6 (Apply, Analyze, Evaluate, Create)
SDG Mapping (SDG 1-17)	SDG 9 (Industry, Innovation & Infrastructure), SDG 16 (Peace, Justice & Strong Institutions)
Industry / Societal Relevance	Reflects real-world Security Operations Centre (SOC) practice — cloud-based, AI/ML-driven cybersecurity monitoring is a core, fast-growing industry function protecting organisational and public digital infrastructure

Problem Statement

Problem / Case / Design Challenge

Organizations increasingly depend on digital technologies, cloud services, IoT devices and online communication systems, and face growing risks from malware, ransomware, phishing, data breaches and unauthorized access. Traditional, isolated security-monitoring tools generate large volumes of alerts that are difficult for security teams to triage manually, causing delayed detection, slow incident response

and higher risk of successful attacks. The assignment requires designing and developing a Smart Network Security Operations Centre (SNSOC) — a centralized, cloud-based cybersecurity platform that continuously monitors network traffic and security logs, applies AI/ML-based analysis to packet and event data, detects threats in real time, and supports automated/assisted incident response.

Requirements and Constraints

- Functional requirement: continuous, centralized monitoring of network traffic, firewalls, servers, applications, endpoints and cloud logs.
- Functional requirement: real-time detection of malware, unauthorized logins, phishing attempts and DDoS/port-scan activity, with automated alert generation.
- Performance requirement: high detection accuracy with a low false-positive rate, and fast incident response time.
- Technical constraint: must integrate heterogeneous data sources and tools (network traffic, system/firewall/IDS/application/cloud logs) into one correlated pipeline.
- Security/privacy constraint: sensitive log and traffic data must be handled in line with information-security standards (e.g., ISO/IEC 27001, ISO/IEC 27002).
- Compliance/standards constraint: architecture must align with recognized frameworks — NIST Cybersecurity Framework, OWASP guidelines, CIS Controls, IEEE software-engineering standards.
- Scalability requirement: the solution must remain effective for small, medium and large organizations and for growing cloud-based network environments.

Student Work

Problem Understanding and Formulation

The problem is the absence of a centralized, intelligent cybersecurity monitoring capability: modern networks generate more security events than manual, siloed tools can analyse, so genuine threats are missed or discovered too late. Expected outcomes are a working SNSOC architecture that ingests logs and network traffic from multiple sources, correlates and analyses them with AI/ML techniques, generates real-time alerts, supports vulnerability assessment and incident response, and produces compliance-ready security reporting.

Available information/data includes network traffic captures, system logs, firewall logs, IDS/IPS alerts, endpoint logs, application logs and cloud logs, drawn from users such as security analysts, SOC operators, administrators and management. Key assumptions made are that historical and simulated security-event data (e.g., normal traffic, unauthorized login, malware activity, phishing attempt, DDoS attack) are representative of real attack classes, and that the organization already operates the underlying network infrastructure (firewalls, routers, endpoints, cloud services) that the SNSOC observes rather than replaces. Constraints that must be satisfied include real-time or near-real-time processing, low false-positive rates, adherence to information-security standards, and scalability across organization sizes.

Application of Course Knowledge

The solution draws directly on Cloud Computing and Big Data Analytics course concepts: large-volume, high-velocity security-log and packet data from many heterogeneous sources (network, firewall, application, cloud) must be collected, stored and processed at scale — a classic big-data ingestion and analytics problem — before AI/ML models can be applied on top. Cloud-based deployment is applied so that data collection, storage and the ML-based threat-detection engine can scale elastically with network size and event volume, rather than being bound to fixed on-premise capacity.

From a security-engineering standpoint, the design applies core concepts of anomaly detection and behavioural analysis (identifying deviations from a normal-traffic baseline), pattern recognition and predictive analysis (recognising known attack signatures and forecasting emerging ones), and event correlation and risk scoring (combining alerts from multiple engines — SIEM correlation, IDS/IPS, vulnerability scanning, malware detection, User and Entity Behaviour Analytics — into a single prioritised risk signal). Machine-learning classification techniques are applied to security events (e.g., distinguishing Normal Traffic from Unauthorized Login, Malware Activity, Phishing Attempt and DDoS Attack) using severity levels (Low/Medium/High/Critical) to drive automated response decisions.

Solution / Design / Methodology

Development Process

The SNSOC was developed through a structured methodology: requirement analysis of existing cybersecurity solutions, monitoring tools, IDS and SOC frameworks; architecture design integrating monitoring, detection, vulnerability-assessment and response components; implementation of the modules; testing against simulated

attack scenarios; and evaluation against cybersecurity performance metrics such as detection accuracy, response time and false-positive rate.

System Architecture (Solution Overview)

The architecture is organised into eleven interconnected layers/modules. A Users Layer (security analyst, SOC operator, administrator, management) interacts with a Data Collection & Log Ingestion layer that gathers network traffic, system logs, firewall logs, IDS/IPS alerts, endpoint logs, application logs and cloud logs. This feeds a Data Processing & Normalization stage (log parsing, filtering, normalization, time synchronization, event enrichment, storage), which in turn feeds the Threat Detection & Analysis Engine — combining AI/ML-based detection (anomaly detection, behaviour analysis, pattern recognition, predictive analysis, threat intelligence) with security-tool integration (SIEM correlation, IDS/IPS, vulnerability scanner, malware detection, UEBA engine) and a Correlation & Risk Scoring step. Identified threats and alerts flow into an Alerting & Notification System (real-time alerts, email/SMS, dashboard notifications, escalation, ticket generation). Supporting components — a threat-intelligence feed, database/data storage, configuration manager, security policies/rules, user management and backup/recovery — feed a Vulnerability Assessment Module, an Incident Response Module (investigation, containment, automated response, forensic analysis, resolution), Dashboards & Visualization, and Reporting & Compliance, all tied together by a Continuous Monitoring & Feedback Loop that improves the models and knowledge base over time. The end output is improved threat detection, reduced response time, stronger network security, better compliance/risk management and operational cost reduction.

Solution Justification

A centralized, AI/ML-driven SNSOC was selected over continuing with isolated, manually monitored tools because it provides real-time visibility across all network layers, early detection of both known and previously unseen threats, automated/assisted incident response, reduced false-positive alerts through intelligent correlation, and a scalable architecture usable by organizations of different sizes — directly addressing the delayed-detection and alert-overload problems identified in the problem statement.

Use of Modern Tools

The following modern engineering/computing tools were used to build and evidence the SNSOC:

Tool / Technology	Role in the SNSOC
Python	Threat-detection algorithms, ML model implementation, security automation scripts, data visualisation and reporting
Wireshark	Packet capture, traffic analysis, network troubleshooting, threat investigation
Snort IDS	Intrusion detection, attack monitoring, security-alert generation, threat identification
SIEM Platform	Log management, event correlation, threat intelligence, security reporting
ML Libraries (NumPy, Pandas, Scikit-learn, TensorFlow)	Intelligent threat analysis and predictive cybersecurity operations
Database Management System	Storage of security logs, incident reports, threat-intelligence data and monitoring records

As implementation evidence, a Python prototype (included in the Appendices) models the core event-classification and alert-generation logic: incoming security events (Normal Traffic, Unauthorized Login, Malware Activity, Phishing Attempt, DDoS Attack) are iterated over, classified by severity level, and either marked "Status: Safe" or trigger "Threat Detected" / "Alert Generated" / "Incident Response Activated" outputs — demonstrating the alerting and incident-response logic described in the architecture.

Results and Validation

The SNSOC was evaluated using simulated network environments and cybersecurity scenarios covering the five modelled event types. The system successfully monitored network traffic, collected security logs, detected suspicious activity and generated alerts in real time. The evaluation showed measurable improvement in threat-detection accuracy, incident-response time, security-event visibility, network-monitoring efficiency, vulnerability identification and overall cybersecurity management, with the AI/ML component specifically improving the system's ability to recognise abnormal behaviour patterns and catch potential threats before they caused significant damage. The prototype run (Appendices) demonstrates the end-to-end flow — event ingestion, iteration/severity

classification, final severity summary (counts of low/medium/high/critical events) and task allocation of each event to the correct response module (Alert & Analysis, Incident Response, Monitoring, or Emergency Response) — validating that the design satisfies the stated functional requirement of centralized, real-time detection and alerting.

Analysis and Engineering Decision

Compared with the traditional alternative of siloed, manually reviewed security tools, the integrated SNSOC trades some implementation complexity (coordinating IDS, SIEM, vulnerability scanning and ML models together) for substantially better detection accuracy, lower false-positive rates and faster response — the priority given the stated problem of alert overload and delayed response. Within the detection engine, combining rule/signature-based tools (Snort IDS, SIEM correlation) with AI/ML-based anomaly and pattern detection was chosen over either approach alone: signature-based tools are precise against known attacks but blind to novel ones, while ML-based behavioural analysis catches unknown/emerging threats at the cost of needing more tuning to control false positives — the correlation and risk-scoring stage was added specifically to reconcile outputs from both and reduce alert fatigue. Real, quantitative performance figures (accuracy, response time, false-positive rate) should be captured from the actual evaluation run and reported here to justify the final configuration chosen.

Broader Considerations

- Standards and professional responsibility: the design applies ISO/IEC 27001 (information security management), the NIST Cybersecurity Framework (Identify–Protect–Detect–Respond–Recover), ISO/IEC 27002 (security controls), IEEE software-engineering standards, OWASP guidelines and CIS Security Controls.
- Economics: centralized, automated monitoring reduces the operational cost of manual triage and reduces the financial impact of successful breaches (data loss, downtime, reputational harm).
- Society and stakeholders: benefits security administrators, IT departments, employees, customers/clients, regulators and researchers by protecting sensitive data and enabling faster, more reliable incident handling (Section 2.3 of the underlying report).
- Safety and reliability: automated incident response must include human-in-the-loop safeguards so that containment actions (e.g., blocking accounts or traffic) do not themselves disrupt legitimate business operations.

- Compliance: the reporting/compliance module supports regulatory audit requirements, helping organizations meet data-protection and cybersecurity regulations.

Conclusion

The Smart Network Security Operations Centre addresses the challenge of fragmented, manually monitored cybersecurity defences by providing a centralized, cloud-based platform that integrates network monitoring, log ingestion, AI/ML-based threat detection, vulnerability assessment, alerting, incident response, dashboards and compliance reporting in a single continuous-feedback architecture. The prototype and simulated evaluation confirm that the design can ingest multiple log/traffic sources, classify events by severity, generate real-time alerts, and route each event to the correct response module. Major findings are that combining signature-based tools with AI/ML behavioural detection, and correlating their outputs before alerting, is key to reducing false positives while still catching novel threats. Limitations include reliance on simulated rather than production-scale traffic for evaluation, and the current design's dependence on a defined set of event categories rather than fully open-ended anomaly discovery. Possible improvements include integrating Deep Learning models, real-time external threat-intelligence feeds, cloud/hybrid-environment security monitoring, and Security Orchestration, Automation and Response (SOAR) capabilities for more autonomous containment.

Student Reflection

What I learned beyond what was directly taught in the classroom

Through this assignment, I learned how cloud computing concepts can be applied to solve a real-world problem. I understood the working of the SaaS model and learned how a cloud-based application can collect, store and analyze information.I also learned how forms, reports and dashboards can be used to create a simple management system using a cloud platform.If additional time and resources were available, the system could be improved by adding automatic notifications, administrator roles, complaint escalation, image upload for complaints and student feedback after complaint resolution.What I would improve with additional time or resources

Given more time, I would integrate Deep Learning-based anomaly detection and live external threat-intelligence feeds so the system could recognise newly discovered attack signatures rather than relying only on the modelled event categories, extend

the architecture to explicitly monitor cloud and hybrid infrastructure, and add Security Orchestration, Automation and Response (SOAR) capabilities so routine containment actions could be executed automatically with minimal analyst intervention.

References

- Stallings, W. — Network Security Essentials: Applications and Standards. Pearson Education.
- Easttom, C. — Computer Security Fundamentals. Pearson.
- NIST Cybersecurity Framework, Version 2.0 — National Institute of Standards and Technology.
- ISO/IEC 27001:2022 — Information Security Management Systems.
- Scarfone, K., & Mell, P. — Guide to Intrusion Detection and Prevention Systems (IDPS). NIST.
- Allen, J. H. — Cybersecurity Operations Handbook. Carnegie Mellon University.
- OWASP Foundation — OWASP Top 10 Web Application Security Risks.
- Splunk Inc. — Security Information and Event Management (SIEM) Best Practices.
- Cisco Systems — Cybersecurity and Network Defense Fundamentals.
- IBM Security — Security Operations Center (SOC) Implementation Guide.
- Python, Wireshark, Snort IDS and SIEM-platform documentation, used as the implementation and analysis tools for this assignment.

Common Assessment Rubric

Assessment Criterion	**Marks**
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	**100**

CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO1	PO1, PO2	L3/L4	10
Application of knowledge	CO1, CO2	PO1, PO2, PO5	L3/L4	20
Solution / Design / Implementation	CO2, CO3	PO2, PO3, PO5	L4/L5/L6	20
Modern tool usage	CO3	PO5	L3/L4	10
Validation	CO4	PO4, PO5	L4/L5	15
Analysis & justification	CO4	PO4	L4/L5	15
Broader considerations	CO5	PO6, PO8	L4/L5	5
Documentation & reflection	CO5	PO9, PO10, PO12	L3/L4	5

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is not required to map to every PO.

Faculty Evaluation & Continuous Improvement CO Attainment

Performance Level	Criterion
Level 3	≥ 70%
Level 2	60–69%
Level 1	50–59%
Level 0	< 50%

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____

Documents to be Maintained

- Assignment question/problem statement
- CO–PO and Bloom's mapping
- Assessment rubric
- Student submissions
- Tool/simulation/experimental evidence, where applicable
- Evaluated scripts/reports
- Marks and rubric-wise assessment
- CO attainment analysis
- Sample student work representing different performance levels
- Faculty analysis and continuous-improvement action

CODE:

```
"""
Smart Network Security Operations Centre (SNSOC)
Event Classification, Alerting and Incident-Response Routing — Prototype
"""

import random
from datetime import datetime, timedelta

# 1. Configuration ------------------------------------------------------

LOG_SOURCES = ["Firewall", "IDS", "Application", "Endpoint", "Cloud"]

EVENT_TYPES = {
    "Normal Traffic":       {"severity": "Low",      "module": "Monitoring"},
    "Port Scan":            {"severity": "Low",      "module": "Alert & Analysis"},
    "Unauthorized Login":   {"severity": "Medium",   "module": "Alert & Analysis"},
    "Phishing Attempt":     {"severity": "Medium",   "module": "Alert & Analysis"},
```

```
    "Malware Activity":     {"severity": "Critical",  "module": "Incident Response"},
    "DDoS Attack":          {"severity": "Critical",  "module": "Incident Response"},
}

ACTIONS = {
    "Low":      "No action required",
    "Medium":   "Alert generated",
    "Critical": "Incident response initiated",
}


# 2. Simulate incoming security events ---------------------------------------

def generate_events(n=10):
    """Simulate n security events collected from network sources."""
    events = []
    for i in range(n):
        event_type = random.choice(list(EVENT_TYPES.keys()))
        events.append({
            "id": i + 1,
            "timestamp":    (datetime.now()    -    timedelta(seconds=random.randint(0,
3600))).strftime("%Y-%m-%d %H:%M:%S"),
            "source": random.choice(LOG_SOURCES),
            "event_type": event_type,
            "ip_address":
f"192.168.{random.randint(0,255)}.{random.randint(0,255)}",
        })
    return events


# 3. Threat detection & analysis engine --------------------------------------
```

```
def classify_event(event):
    """Attach severity, action and target response module to a raw event."""
    meta = EVENT_TYPES[event["event_type"]]
    severity = meta["severity"]
    event["severity"] = severity
    event["action"] = ACTIONS[severity]
    event["response_module"] = meta["module"] if severity != "Low" or
event["event_type"] != "Normal Traffic" else "Monitoring"
    return event


def analyze_events(events):
    return [classify_event(e) for e in events]


# 4. Alerting & reporting ----------------------------------------------------

def summarize(events):
    summary = {"Low": 0, "Medium": 0, "Critical": 0}
    for e in events:
        summary[e["severity"]] += 1
    return summary


def print_event_log(events):
    print(f"{'Event':22}{'Severity':12}{'Action'}")
    print("-" * 70)
    for e in events:
        print(f"{e['event_type']:22}{e['severity']:12}{e['action']}")


def print_summary(summary, total):
```

```
    print("\nFinal Result: Security Summary")
    print(f"  Total events analyzed : {total}")
    print(f"  Low severity events   : {summary['Low']}")
    print(f"  Medium severity events: {summary['Medium']}")
    print(f"  Critical events       : {summary['Critical']}")


def print_module_routing(events):
    print("\nTask Allocation to Response Modules")
    for e in events:
        print(f"    Event {e['id']:02d} ({e['event_type']:<18}) -> Assigned to: {e['response_module']}")


# 5. Main execution ------------------------------------------------------------

def main():
    print("SMART NETWORK SECURITY OPERATIONS CENTRE (SNSOC)\n")

    print("1. Input: Security events collected from network sources\n")
    events = generate_events(10)

    print("2. Iteration Process: Threat Analysis\n")
    events = analyze_events(events)
    print_event_log(events)

    summary = summarize(events)
    print_summary(summary, len(events))

    print_module_routing(events)

    print("\nProcess finished with exit code 0")
```

```
if __name__ == "__main__":
    main()
```

Output